

AlgoRanker

Visualizador Interativo de Algoritmos de Ordenação

Projeto Acadêmico: Algoritmos e Complexidade

Aluno: Tupi Guedes Ribas | **Prof.** MSc Heleno Cardoso

Visão Geral do Projeto

Objetivo

Desenvolver uma Single Page Application (SPA) que permite visualizar, em tempo real, o comportamento e a performance de algoritmos de ordenação aplicados a um ranking de jogadores.

Escopo

O sistema gerencia um cadastro dinâmico de pontuações, aplica algoritmos de ordenação (Bubble, Merge, Quick) e apresenta visualmente as comparações e trocas de elementos.

Stack Tecnológica

A aplicação foi construída utilizando uma arquitetura moderna baseada em componentes, focada em performance de renderização.

- ✓ **React e TypeScript:** Tipagem estática e reatividade para controle preciso do estado da animação.
- ✓ **Vite:** Build tool de alta performance.
- ✓ **Tailwind CSS:** Estilização utilitária para layout responsivo.
- ✓ **Docker:** Containerização via Dockerfile multi-stage.
- ✓ **Fly.io:** Deploy contínuo via GitHub Actions.

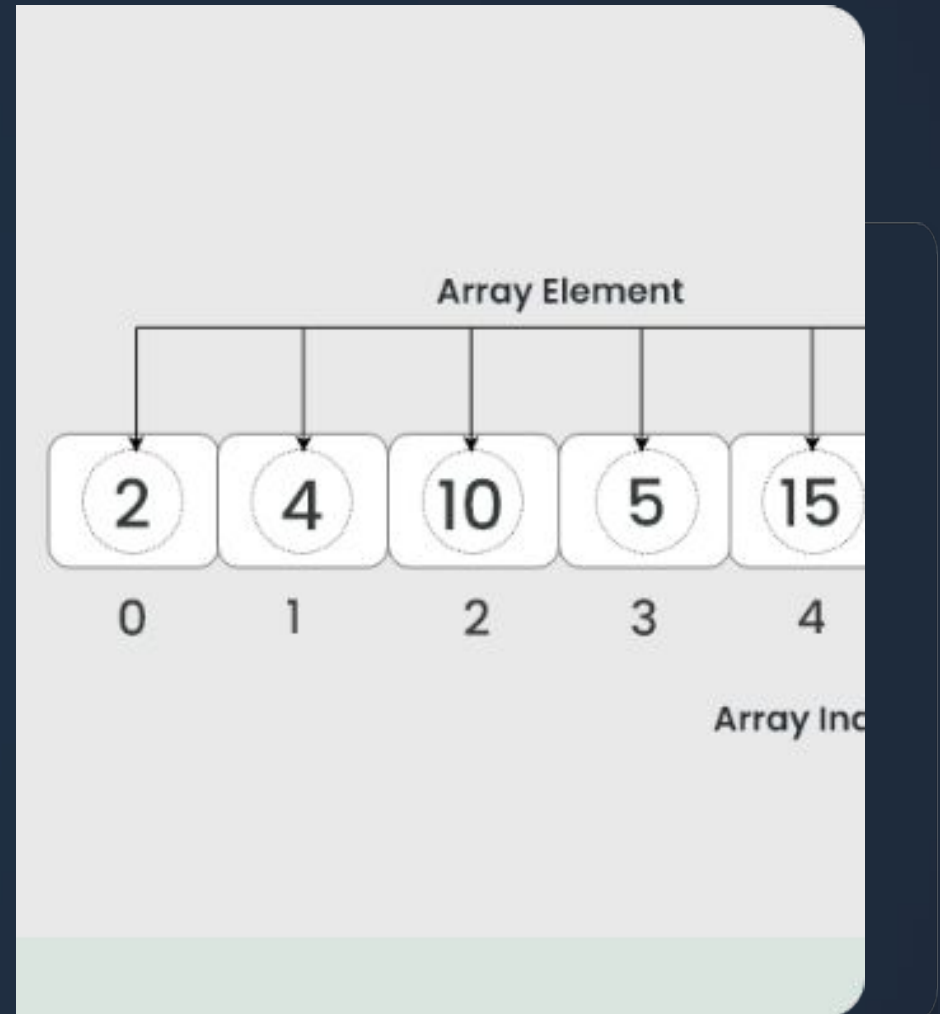
Estrutura de Dados: Array

Por que Arrays?

A estrutura principal é um vetor de objetos Player. A escolha se deve a:

- ✓ **Acesso $O(1)$:** Essencial para leitura rápida durante as comparações visuais.
- ✓ **Cache Locality:** Elementos contíguos na memória melhoram a performance real.
- ✓ **Compatibilidade:** Algoritmos clássicos como QuickSort operam nativamente sobre índices.

```
interface Player {  
  id: number;  
  name: string;  
  score: number;  
}
```



1. Bubble Sort

```
export const getBubbleSortAnimations = (array) => { const n = array.length; //  
  Loop Externo: O(n) for (let i = 0; i < n - 1; i++) { // Loop Interno: O(n - i)  
    for (let j = 0; j < n - i - 1; j++) { // Operação Constante: O(1) if  
      (array[j].score < array[j + 1].score) { swap(array, j, j + 1); } } } };
```

COMPLEXIDADE DE TEMPO

$O(n^2)$

Ocorre devido aos dois loops aninhados. O loop externo executa n vezes e o interno executa aproximadamente n vezes no pior caso.

CÁLCULO

$$(n-1) + (n-2) + \dots + 1 \approx n^2/2$$

2. Merge Sort

```
function mergeSortHelper(arr, start, end) { // Base da recursão if (start ≥  
end) return; const middle = Math.floor((start + end) / 2); // Divisão: 2 *  
T(n/2) mergeSortHelper(arr, start, middle); mergeSortHelper(arr, middle + 1,  
end); // Conquista (Merge): O(n) doMerge(arr, start, middle, end); } function  
doMerge(...) { // Cria arrays auxiliares (Custo de Espaço) const leftHalf =  
mainArray.slice(...); const rightHalf = mainArray.slice(...); // ...lógica de  
intercalação... }
```

COMPLEXIDADE DE TEMPO

$O(n \log n)$

Consistente em todos os casos (melhor, médio, pior).

A altura da árvore de recursão é $n \log n$ e cada nível custa n para o merge.

RELAÇÃO DE RECORRÊNCIA

$T(n) = 2T(n/2) + O(n)$

3. Quick Sort

```
function partition(arr, low, high) { // Pivô fixo no último elemento const pivot
= arr[high].score; let i = low - 1; // Loop Único: O(n) for (let j = low; j <
high; j++) { if (arr[j].score > pivot) { i++; swap(arr, i, j); } } swap(arr, i +
1, high); return i + 1; } function quickSortHelper(...) { // Chamadas recursivas
dependem do pivô quickSortHelper(arr, low, pi - 1); quickSortHelper(arr, pi + 1,
high); }
```

CASO MÉDIO (ESPERADO)

$O(n \log n)$

Ocorre quando o pivô divide o array em partes balanceadas.

PIOR CASO

$O(n^2)$

Como o pivô é fixo no fim (`arr[high]`), se o array já estiver ordenado, a partição será desbalanceada (O e $n-1$ elementos), degradando a performance.

Resultados e Deploy

O projeto cumpre os requisitos de análise de complexidade e implementação web moderna.

Status: Online 

 **App:** <https://algoranker.fly.dev/>

 **Repo:** github.com/tupi/algoranker

AlgoRanker

Ordenar por:

QuickSort

MergeSort

BubbleSort

Adicionar Jogador

Resetar

Estatísticas

8

Jogadores

100

Pont. Máxima

64

Pont. Mínima

83.5

Média

Apresentação do Site

